Read a Range of Data - LIMIT and OFFSET

Now you know how to get a single row with `.one()`, `.first()`, and `session.get()`.

And you also know how to get multiple rows while filtering them using `.where()`.

Now let's see how to get only a **range of results**.

hero			
id	**name**	**secret_name**	**age**
1	Deadpond	Dive Wilson	null
2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48
4	Tarantula	Natalia Roman-on	32
5	Black Lion	Trevor Challa	35
6	Dr. Weird	Steve Weird	36
7	Captain North America	Esteban Rogelios	93

Create Data

We will continue with the same code as before, but we'll modify the `select_heroes()` function to simplify the example and focus on what we want to achieve here.

Again, we will create several heroes to have some data to select from:

Python 3.10+

```python
# Code above omitted 👆

def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
```

```
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).limit(3)
        results = session.exec(statement)
        heroes = results.all()
        print(heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Review Select All

This is the code we had to select all the heroes in the `select()` examples:

Python 3.10+

```
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        heroes = results.all()
        print(heroes)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        heroes = results.all()
        print(heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

But this would get us **all** the heroes at the same time, in a database that could have thousands, that could be problematic.

Select with Limit

We currently have 7 heroes in the database. But we could as well have thousands, so let's limit the results to get only the first 3:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
```

```
    with Session(engine) as session:
        statement = select(Hero).limit(3)
        results = session.exec(statement)
        heroes = results.all()
        print(heroes)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).limit(3)
        results = session.exec(statement)
        heroes = results.all()
        print(heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

The special **select** object we get from `select()` also has a method `.limit()` that we can use to limit the results to a certain number.

In this case, instead of getting all the 7 rows, we are limiting them to only get the first 3.

hero			
id	**name**	**secret_name**	**age**
1	Deadpond	Dive Wilson	null
2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48
4	Tarantula	Natalia Roman-on	32
5	Black Lion	Trevor Challa	35
6	Dr. Weird	Steve Weird	36
7	Captain North America	Esteban Rogelios	93

Run the Program on the Command Line

If we run it on the command line, it will output:

```bash
```

Great! We got only 3 heroes as we wanted.

> **Tip**
>
> We will check out that SQL code more in a bit.

Select with Offset and Limit

Now we can limit the results to get only the first 3.

But imagine we are in a user interface showing the results in batches of 3 heroes at a time.

> **Tip**
>
> This is commonly called "pagination". Because the user interface would normally show a "page" of a predefined number of heroes at a time.
>
> And then you can interact with the user interface to get the next page, and so on.

How do we get the next 3?

hero			
id	**name**	**secret_name**	**age**
1	Deadpond	Dive Wilson	null
2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48
4	Tarantula	Natalia Roman-on	32
5	Black Lion	Trevor Challa	35
6	Dr. Weird	Steve Weird	36
7	Captain North America	Esteban Rogelios	93

We can use `.offset()`:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).offset(3).limit(3)
        results = session.exec(statement)
        heroes = results.all()
        print(heroes)
```

```
# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).offset(3).limit(3)
        results = session.exec(statement)
        heroes = results.all()
        print(heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

The way this works is that the special **select** object we get from `select()` has methods like `.where()`, `.offset()` and `.limit()`.

Each of those methods applies the change in the internal special select statement object, and also **return the same object**, this way, we can continue using more methods on it, like in the example above that we use both `.offset()` and

`.limit()`.

Offset means "skip this many rows", and as we want to skip the ones we already saw, the first three, we use `.offset(3)`.

Run the Program with Offset on the Command Line

Now we can run the program on the command line, and it will output:

```bash
```

Select Next Batch

Then to get the next batch of 3 rows we would offset all the ones we already saw, the first 6:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).offset(6).limit(3)
        results = session.exec(statement)
        heroes = results.all()
        print(heroes)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).offset(6).limit(3)
        results = session.exec(statement)
        heroes = results.all()
        print(heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

The database right now has **only 7 rows**, so this query can only get 1 row.

hero			
id	**name**	**secret_name**	**age**
1	Deadpond	Dive Wilson	null
2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48
4	Tarantula	Natalia Roman-on	32
5	Black Lion	Trevor Challa	35
6	Dr. Weird	Steve Weird	36
7	Captain North America	Esteban Rogelios	93

But don't worry, the database won't throw an error trying to get 3 rows when there's only one (as would happen with a Python list).

The database knows that we want to **limit** the number of results, but it doesn't necessarily have to find that many results.

Run the Program with the Last Batch on the Command Line

And if we run it in the command line, it will output:

```bash
```

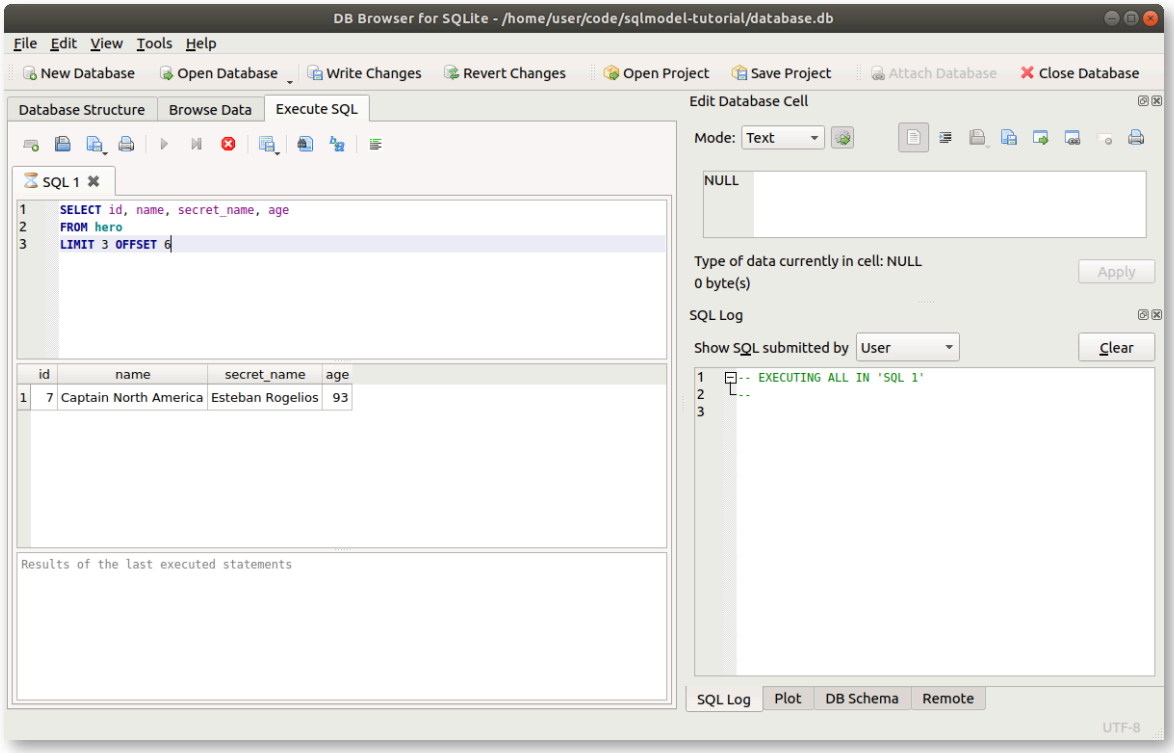
SQL with LIMIT and OFFSET

You probably noticed the new SQL keywords `LIMIT` and `OFFSET`.

You can use them in SQL, at the end of the other parts:

```
SELECT id, name, secret_name, age
FROM hero
LIMIT 3 OFFSET 6
```

If you try that in **DB Browser for SQLite**, you will get the same result:



Combine Limit and Offset with Where

Of course, you can also combine `.limit()` and `.offset()` with `.where()` and other methods you will learn about later:

Python 3.10+

```
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age > 32).offset(1).limit(2)
        results = session.exec(statement)
        heroes = results.all()
        print(heroes)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age > 32).offset(1).limit(2)
        results = session.exec(statement)
        heroes = results.all()
        print(heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Run the Program with Limit, Offset, and Where on the Command Line

If we run it on the command line, it will find all the heroes in the database with an age above 32. That would normally be 4 heroes.

But we are starting to include after an offset of 1 (so we don't count the first one), and we are limiting the results to only get the first 2 after that:

```bash
```

Recap

Independently of how you filter the data with `.where()` or other methods, you can limit the query to get at maximum some number of results with `.limit()`.

And the same way, you can skip the first results with `.offset()`.